

anvil: A Code Transformation Framework for the R Language

by Sebastian Fischer, Daniel Falbel, Thomas Kalinowski, Niko German, Martin Binder, and Bernd Bischl

Abstract The anvil package provides a composable code transformation framework for the R language. It currently supports (backward-mode) automatic differentiation and execution is handled by jit-compilation via the OpenXLA compiler framework, which has support for different hardware backends (CPU, GPU, etc.). Together, these features allow to implement numerical algorithms efficiently in pure R. All code transformations are based on first tracing R code into a computational graph via abstract evaluation. This allows to combine jit-compilable operations with standard R code, the latter being “staged out” during compilation. The package is also designed to be extendable, meaning it is possible to add new primitive operations or code transformations. By being mostly written in R, the package is also easy to contribute to.

0.1 Introduction

All statistical and machine learning models fundamentally rely on some form of array or matrix operations. Even though R as an interpreted language has performance limitations, relying on optimized matrix operations can often be fast enough for many use cases. However, there are cases where the performance is not sufficient and one needs to resort to compiled languages such as C++. Furthermore, one might additionally want to have access to GPU acceleration or automatic differentiation for gradient based optimization.

To address this, we introduce the **anvil**, which is a composable code transformation framework for the R language. Its primary goal is to offer a set of primitive operations that can be jit-compiled, differentiated, and executed on different hardware backends. It therefore allows to implement numerical algorithms efficiently in pure R. For jit-compilation, we leverage the OpenXLA ecosystem, which is a compiler ecosystem. Part of this project is the StableHLO language, which is a strongly typed language for representing array programs, as well as the PJRT (Pluggable Jit RunTime) library, which provides a common runtime for executing compiled programs. Components of **anvil** are the **stablehlo** and **pjrt** packages. The former allows to easily create StableHLO programs in R, while the latter interfaces the PJRT C library and handles things like compilation, data allocation and execution. Besides the OpenXLA backend, there is also an alternative, experimental, backend that compiles R code into Fortran code via the **quicksr** package. Because the **anvil** package is primarily written in R, it is easy to extend and contribute to.

The closest related work is the JAX python library, from which various design elements are inspired or adopted. Specifically, we are using a similar functional design and also use a tracing-based approach for code transformations such as automatic differentiation. Also for other design elements (like the handling of nested data structures) we following the JAX approach, which are often a natural choice, as JAX is also built on top of XLA. For the **stablehlo** package, we have adopted ideas from **mlr3torch**, which offers a similar graph-building interface for creating neural networks. Also the tracing-based approach in **anvil** is similar to the one in **mlr3torch**. What PipeOpTorch objects are in **mlr3torch** are akin to the primitive graph-building functions in **anvil**. For the design of the **pjrt** package, we have borrowed from the gopjrt library (cite) which offers a go interface to PJRT.

The OpenXLA project (cite) is an open-source compiler ecosystem for machine learning, originally developed at Google as part of TensorFlow and later released as a standalone project. Its goal is to decouple numerical computing frameworks from hardware-specific compilers by defining two standardized interfaces. The first is the StableHLO language (cite), a portable intermediate representation for tensor programs. StableHLO is built on MLIR (Multi-Level Intermediate Representation), which is itself part of the LLVM compiler

infrastructure. MLIR provides a framework for defining and transforming intermediate representations at multiple levels of abstraction, which makes it well suited for representing and optimizing numerical programs. The second interface is PJRT (Pluggable JIT RunTime), which defines a common API for the essential operations of a JIT-compiled tensor computing framework: buffer management, compilation, and execution. PJRT follows a plugin architecture, where each hardware vendor can provide a shared library that implements the PJRT interface for their specific hardware. This means that any framework that emits StableHLO and uses PJRT for execution can automatically target any hardware for which a PJRT plugin exists, including CPUs, NVIDIA GPUs, and Google TPUs. Currently, the main compiler consuming StableHLO programs is the XLA compiler, but alternatives such as the IREE compiler (cite) also exist.

The main technical problem **anvil** tries to solve are to go beyond the limitations of the **torch** package. The **torch** package offers a collection of highly optimized tensor operations that call into the LibTorch C++ library, as well as various modules for building and training neural networks. As such, it also has support for backward-mode automatic differentiation and can be run on different hardware backends, including CPU and GPU. However, its default mode is to be executed eagerly, which has performance implications. For example, all intermediate tensors that are calculated during a neural network forward pass are stored in R and have to eventually be cleaned up by the R garbage collector. While **torch** offers a way to JIT-compile code via tracing code and converting it to TorchScript, this feature is in maintenance mode and has no support for automatic differentiation. Furthermore, when operating on small tensors with many operations (such as a loop), the continuous switching between R and the kernels can be costly and there is no optimization across the different operations.

There are also other R packages that offer GPU acceleration and automatic differentiation for tensor operations. These are **tensorflow** and **keras3**, two deep learning frameworks that interface the TensorFlow Python runtime via **reticulate**. Similarly, it is possible to use Python's JAX library via **reticulate**. While these packages provide access to mature GPU ecosystems with extensive operator libraries, they require managing a Python environment and expressing computations in framework-specific tensor APIs rather than in R.

Beyond these deep learning frameworks, there have been several attempts at general-purpose GPU programming in R. Packages such as **gputools**, **gpuR**, and **GPUMatrix** provided GPU-resident matrix types and operations, but have all since been archived, reflecting the difficulty of maintaining GPU toolchains on CRAN. The **OpenCL** package takes a lower-level approach, allowing users to write and execute OpenCL kernels directly from R, but requires authoring GPU kernels manually rather than compiling R code. The ability to trace and compile *user-written R code* to GPU remains rare. **anvil** targets this niche by tracing R functions into a computational graph, lowering to StableHLO, and executing via PJRT, which provides a hardware-agnostic runtime with plugins for CPU, NVIDIA GPU, and other backends.

Other R packages exist that also allow compiling R code. The most recent example is **quicksr**, which transpiles R code into Fortran code and then compiles it. There are various differences between the two approaches. First, **quicksr** has no support for automatic differentiation and currently only runs on CPU. Besides, **quicksr** does not establish a new API, but instead aims to be a drop-in replacement for a set of R functions. This works because it is implemented via a static analysis approach, so the package does not need control over the functions that can be compiled as is the case for the tracing approach followed in **anvil**. This also means that **quicksr** does not allow mixing “non-compilable” R code with compilable code, as is possible for **anvil**, where the “non-compilable” code is staged out during compilation. The tracing approach is more flexible, but also requires the user to be more careful about the code that they are writing. Similar to **quicksr**, there is also **ast2ast**, which translates a subset of R code into C++ by analyzing it. Then, there is of course the **compiler** package itself (cite) that compiles arbitrary R code into byte-code that is then executed by the R Virtual Machine. While this leads to considerable speedups, its optimization potential is limited by the design of the R language (Morandat et al., 2012), where anything can happen

at any time. This generally highlights the trade-off between expressiveness and performance. In **anvil**, we are focussing on a rather narrow set of numerical operations, which thus allows for aggressive optimizations and highly efficient execution.

There are various other packages that share similarities with **anvil**, be it in compilation of R code, automatic differentiation, a systematic tensor programming toolkit or execution on accelerators like GPUs. There is also the **nimble** package, which allows to define Bayesian models in a DSL and compile them to C++ code. It has a different focus than **anvil** and, for example, does not support automatic differentiation or GPU execution. Similarly, **rstan** provides an R interface to the Stan probabilistic programming language (cite), which compiles statistical models to C++ and supports automatic differentiation for Hamiltonian Monte Carlo and variational inference. While Stan shares several technical capabilities with **anvil** — compilation, automatic differentiation, and efficient execution — there are important differences. First, Stan requires models to be written in a separate, C++-like language, whereas **anvil** allows users to write R code that can be freely mixed with standard R. Second, Stan’s AD and compiler are tightly integrated into its inference engine (MCMC, variational inference, optimization) and are not exposed as composable building blocks. In **anvil**, `gradient()` returns a regular function that can be called within other traced functions, enabling patterns like embedding gradient computations inside compiled training loops. Third, **anvil** targets general-purpose numerical computing and can run on different hardware backends (CPU, GPU) via the PJRT plugin system, while Stan primarily targets CPU execution. Finally, **anvil** is designed to be extensible with new primitives, transformations, and backends, whereas Stan is a more self-contained system.

Then, there are **odin** and **rxode2**, but of which focus on differential equations and compile a small subset of R code via translation to C, similar to **quicker**.

There are also further projects that go into a similar direction, but are either proof of concepts or not actively developed anymore. One of these projects is the FastR project, which aims for a drop-in replacement for the R interpreter (cite) which is based on the GraalVM project (cite), but there is little activity on the project. There is also vapour, which is a typed superset of R that is transpiled to R, also without active development.

Then there is **r2c**, which is a proof of concept implementation that is similar to **quicker**, but translates R code to C instead of Fortran.

Several R packages provide automatic differentiation (AD), differing in AD mode, derivative order, execution model, and the scope of functions that can be differentiated.

A first category consists of pure R packages that implement forward-mode AD via operator overloading on dual number objects. The most comprehensive of these is **anbla**, which supports arbitrary-order derivatives (gradients, Hessians, and higher-order tensors) via composition of a single derivative operator `D`, and works through standard R control flow including loops and branches. Other forward-mode packages include **salad**, **dual**, **madness**, and **ADtools**, which vary in their coverage of base R functions and matrix operations. The archived **dfdr** package took a different approach, constructing derivative expressions via source transformation, but was limited to single-expression functions with few supported operations. A common limitation of all these packages is that the differentiated code is still executed by the R interpreter, which limits performance for large-scale problems. Furthermore, forward-mode AD has a computational cost that scales with the number of input parameters, making it less efficient than reverse mode for the common case of many parameters and a scalar objective.

A second category consists of packages where AD is provided by an external C++ library and R serves as a front-end. **TMB** and **RTMB** use the CppAD/TMBad tape-based AD system for reverse-mode differentiation, primarily targeting likelihood-based inference with random effects. TMB requires the objective to be written in C++ templates, while RTMB allows writing it in a subset of R, with both producing compiled executables that run outside of R. **rstan** provides access to Stan Math’s AD system, which supports forward, reverse, and mixed-mode differentiation, but only for models expressed in the Stan language. **nimble** includes an AD system for its model graphs and **nimbleFunctions**, enabling gradient-based

methods like HMC (via **nimbleHMC**). **torch** uses LibTorch’s tape-based autograd for reverse-mode differentiation of tensor operations. These frameworks offer fast execution and mature reverse-mode AD, but require expressing the computation in a framework-specific DSL or API rather than plain R code.

anvil occupies a middle ground. Like the pure R forward-mode packages, the user writes R code using a set of provided operations. However, like the external frameworks, the entire computation — including the gradient — is compiled and executed outside of R via the XLA compiler. **anvil** currently implements backward-mode AD, which is more efficient than forward mode for the typical case of many inputs and scalar output. A current limitation compared to **nabla** or Stan Math is that **anvil** only supports first-order gradients and does not yet provide Hessians or higher-order derivatives.

The structure of the paper is as follows: First, we will briefly review the OpenXLA compiler ecosystem, including the StableHLO language and the PJRT library, as well as the two corresponding R packages **stablehlo** and **pjrt**. In the subsequent part, we will describe the design of the main **anvil** package. Then, we will demonstrate the package’s abilities using a simple example. Afterwards, we will discuss how to extend the package. Then, we will include some small CPU runtime benchmarks to illustrate the performance of the package. Finally, we will conclude by discussing current limitations and directions for future work.

0.2 The OpenXLA Ecosystem

One of the goals of the OpenXLA project is to establish standardized interfaces between numerical frameworks and execution engines. This is achieved via two components:

1. The “Pluggable Jit RunTime” (PJRT) which defines a common interface for execution engines.
2. The StableHLO language, which serves as a common intermediate representation for numerical computations.

This generally turns an $n \times m$ problem (n frontends need to run on m different hardware backends) into a $n + m$ problem.

```
%%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#ffffff', 'primaryBorderColor': '#333333'}}
graph TB
```

```
    subgraph left ["Without OpenXLA (n × m)"]
        direction LR
        subgraph F1 [" Frontends "]
            A1[JAX]
            A2[PyTorch]
            A3[TensorFlow]
            A4[anvil]
        end
        subgraph B1 [" Backends "]
            C1[CPU]
            C2[GPU]
            C3[TPU]
        end
    end
    A1 --> C1
    A1 --> C2
    A1 --> C3
    A2 --> C1
    A2 --> C2
    A2 --> C3
    A3 --> C1
    A3 --> C2
    A3 --> C3
```

```

    A4 --> C1
    A4 --> C2
    A4 --> C3
end

subgraph right ["With OpenXLA (n + m)"]
  direction LR
  subgraph F2 [" Frontends "]
    D1[JAX]
    D2[PyTorch]
    D3[TensorFlow]
    D4[anvil]
  end
  end
  subgraph O [" OpenXLA "]
    IR[StableHLO]
    RT[PJRT]
  end
  end
  subgraph B2 [" Backends "]
    G1[CPU]
    G2[GPU]
    G3[TPU]
  end
  end
  D1 --> IR
  D2 --> IR
  D3 --> IR
  D4 --> IR
  IR --> RT
  RT --> G1
  RT --> G2
  RT --> G3
end

```

StableHLO

The StableHLO language is an intermediate representation (IR) for representing numerical computations. Below, we see such a simple program that takes f32 tensors of shape 2x2 and multiplies them.

```

func @main(%arg0: tensor<2x2xf32>, %arg1: tensor<2x2xf32>) -> tensor<2x2xf32> {
  %0 = "stablehlo.multiply"(%arg0, %arg1) : (tensor<2x2xf32>, tensor<2x2xf32>) -> tensor<2x2xf32>
  return %0 : tensor<2x2xf32>
}

```

It is expressed in the MLIR (Multi-Level Intermediate Representation) format, which is a low-level intermediate representation for representing numerical computations. It is in single static assignment (SSA) form, meaning that each variable is assigned to exactly once. Furthermore, shapes are static, meaning that

The language operates primarily on tensors and a tensor is composed of a shape and a data type. The body of a function is a sequence of primitive operations. Moreover, StableHLO is in single static assignment (SSA) form, meaning that each variable is assigned to exactly once. Also, values are immutable, so no in-place updates are possible (although compilers can of course optimize this). For a description of the language, see the [StableHLO specification](#), which also contains a description of the around 100 supported primitive operations.

The [stablehlo](#) package provides a R interface to the StableHLO language. The motivation behind implementing this in pure R is to make the installation of the package easier as

compared to interfacing the C++ implementation that requires LLVM. Below, we will re-create the above program using the [stablehlo](#) package.

```
library(stablehlo)
func <- local_func("main")
func

#> func.func @main () -> {
#>
#> }

x <- hlo_input("x", "f32", shape = c(2, 2), func = func)
x

#> Variable %x in:
#> func.func @main (%x: tensor<2x2xf32>) -> {
#>
#> }

y <- hlo_input("y", "f32", shape = c(2, 2), func = func)
y

#> Variable %y in:
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> {
#>
#> }

z <- hlo_multiply(x, y)
z

#> Variable %0 in:
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> {
#> %0 = "stablehlo.multiply" (%x, %y): (tensor<2x2xf32>, tensor<2x2xf32>) -> (tensor<2x2xf32>)
#> }

f <- hlo_return(z)
hlo_string <- repr(f)
cat(hlo_string, "\n")

#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = "stablehlo.multiply" (%x, %y): (tensor<2x2xf32>, tensor<2x2xf32>) -> (tensor<2x2xf32>)
#> "func.return"(%0): (tensor<2x2xf32>) -> ()
#> }
#>
```

We won't cover this in more detail, but refer to the corresponding [vignette](#) for more details. As far as the [anvil](#) package is concerned, the most important thing is that we can easily create StableHLO programs in pure R.

PJRT

In order to do something with the above program, we need to be able to compile it, create data for it, and run it. This is possible via the [pjrt](#) package, which interfaces the PJRT C++ library. The PJRT library itself is header-only and only defines the interface. A specific implementation of the interface (e.g. for CPU or NVIDIA GPU) is accessed by downloading prebuilt shared libraries and loading them dynamically. Just like for the [stablehlo](#) package,

we only show the core functionality of `pjrt` here and refer to the corresponding [vignette](#) for more details.

Below, we start by creating two `PJRTBuffer` objects from R data. Just like the tensors in the above `StableHLO` program, buffers have a shape and a data type. Shapes of length 0 are used to represent scalars. Buffers can also live in different devices, the CPU being the default.

```
library(pjrt)
x <- pjrt_buffer(1:4, dtype = "f32", device = "cpu")
x

#> PJRTBuffer
#> 1
#> 2
#> 3
#> 4
#> [ CPUf32{4} ]

y <- pjrt_buffer(5:8, dtype = "f32", device = "cpu")
y

#> PJRTBuffer
#> 5
#> 6
#> 7
#> 8
#> [ CPUf32{4} ]
```

In order to run the above program, we convert the string representation into a `PJRTProgram` and then compile it into a `PJRTLoadedExecutable` object. Then, we can execute the program by passing the `PJRTBuffer` objects to the `PJRTLoadedExecutable` object.

```
program <- pjrt_program(src = hlo_string, format = "mlir")
program

#> PJRTProgram(format=mlir, code_size=218)
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = "stablehlo.multiply" (%x, %y): (tensor<2x2xf32>, tensor<2x2xf32>) -> (tensor<2x2xf32>)
#> "func.return"(%0): (tensor<2x2xf32>) -> ()
#> }
#>
#>
```

```
executable <- pjrt_compile(program)
executable

#> <PJRTLoadedExecutable>
```

We can now run the program by passing the `PJRTBuffer` objects to the `PJRTLoadedExecutable` object.

```
out <- pjrt_execute(executable, x, y)
```

Conversion back to R is possible via the `as_array()` function.

```
as_array(out)
```

```
#>      [,1] [,2]
#> [1,]    5  12
#> [2,]   21  32
```

Because PJRTBuffer objects are external pointers that are not handled natively by R's serialization mechanism, we provide a custom serialization method via the [safetensors](#) R package.

```
library(safetensors)
tmp <- tempfile(fileext = ".safetensors")
safetensors::safe_save_file(list(x = out), tmp, framework = "pjrt")
reloaded <- safetensors::safe_load_file(tmp, framework = "pjrt")
reloaded$x

#> PJRTBuffer
#>    5 12
#>   21 32
#> [ CPUf32{2x2} ]
```

Next, we will cover the design of the [anvil](#) package.

0.3 Package Design and Features

The [anvil](#) package implements a code transformation framework for the R language. In general, there are three types of transformations:

1. Transforming R code into a computational graph.
2. Transforming a computational graph into another computational graph.
3. Transforming a computational graph into an executable.

We will now cover these three types of transformations in more detail. Then, we explain how these transformations are wrapped in the main user interface.

Core Graph Transformations

Tracing R code into a computational graph When it comes to translating code into other code, there are in principle two approaches:

1. Static analysis of the code.
2. Dynamically executing the code with “special values” that record certain operations and build up a (partial) representation of the program.

In [anvil](#), we are following the latter approach, because it allows to seamlessly combine standard R code with jit-compilable operations. The function that takes in an R function and returns a computational graph is called `trace_fn()`. In order for this to be useful, the function needs to call into `anvil` primitive functions. These primitive functions are similar to the stableHLO operations like `hlo_add` from earlier.

In the [anvil](#) package these are called `nv1_<op>` functions. However, they are not exported directly, but in thin wrappers like `nv_add`, which perform some convenience transformations (like type casting) that make them more ergonomic to use. These thin wrappers call into the primitive functions, however. The primitive functions are essentially graph building functions, also similar to the `hlo_add` function from earlier. Below, we create an R function that takes in two values and multiplies them. Note that we could have also used the `*` infix operator, but we want to illustrate the use of the `nv_multiply` function.


```
library(anvil)
multiply_r <- function(x, y) {
  nv_mul(x, y)
}
```

We can now trace this function into a computational graph. At the beginning, this will create a `anvil::GraphDescriptor` that primitive functions can attach to to build up the graph. The types for the inputs are inferred from the example arguments. The `trace_fn` transformation uses these example inputs to obtain abstract `GraphBox` objects, which are passed around during the evaluation of the function. They essentially define the type (shape and data type), and uniquely identify a specific `GraphValue` within a specific `GraphDescriptor`. Note that `trace_fn` also takes in non-tensor values, which are evaluated like normal R code.

```
graph <- trace_fn(multiply_r, list(x = nv_scalar(1, "f32"), y = nv_scalar(5, "f32")))
graph

#> <AnvilGraph>
#> Inputs:
#>   %x1: f32[]
#>   %x2: f32[]
#> Body:
#>   %1: f32[] = mul(%x1, %x2)
#> Outputs:
#>   %1: f32[]
```

Once the evaluation of `multiply_r` with the special `GraphBox` objects is complete, the `GraphDescriptor` is converted into a `Graph` object. The only difference between them is that `GraphDescriptor` does some book-keeping during program creation which is not needed anymore afterwards.

From the printed output of the graph we can see that a graph has:

1. Inputs: `GraphValues` that are the inputs to the graph.
2. Constants: `GraphValues` that are constants in the graph.
3. Body: A list of `PrimitiveCall` objects, which represent the primitive operations that make up the graph.
4. Outputs: `GraphValues` that are the outputs of the graph.
5. In-tree and Out-tree: Because Graphs only take in tensor values, but we want to support nested data structures, the in-tree and out-tree define how to (un-)flatten the inputs and outputs of the graph.

STATIC SHAPES!!!

Transforming a Graph into another Graph Once our program no longer consists arbitrary R code, but is in a standardized form, we can transform the program by creating another `Graph` object. Currently, the only example for such a type of transformation is the gradient transformation, which is implemented in the `transform_gradient()` function. It takes in a `Graph` and the names of the arguments to compute the gradient with respect to.

```
gradient_graph <- transform_gradient(graph, wrt = c("x", "y"))
```

The `anvil` package does not put any restrictions on *how* such a transformation is implemented. Usually, however, such a transformation needs to store information for how to transform each `primitiveF`. For this reason, the `anvil::Primitive` objects have a `@rules` field that can be populated. For example, the backward rule for the addition primitive is implemented as follows:

```

prim("add")["backward"]

#> function (inputs, outputs, grads, .required)
#> {
#>   grad <- grads[[1L]]
#>   list(if (.required[[1L]]) grad, if (.required[[2L]]) grad)
#> }
#> <bytecode: 0x108a91728>
#> <environment: namespace:anvil>

```

The transformation function `transform_gradient` basically walks the graph backwards and applies the backward rules to each primitive, producing a Graph that contains the forward and backward pass.

Automatic Differentiation The `transform_gradient()` function implements backward-mode automatic differentiation (also known as reverse-mode AD). Given a Graph representing a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, it produces a new Graph that computes both the original output and the gradients of the output with respect to specified inputs.

More concretely, consider a computational graph as a sequence of primitive operations $v_k = g_k(v_{\text{pa}(k)})$, where $v_{\text{pa}(k)}$ denotes the parent values (inputs) of operation k . The goal is to compute $\bar{v}_i = \frac{\partial f}{\partial v_i}$ for each intermediate value v_i . Starting from $\bar{f} = 1$, the backward pass walks the graph in reverse topological order and accumulates gradients via the chain rule:

$$\bar{v}_i = \sum_{k: i \in \text{pa}(k)} \bar{v}_k \cdot \frac{\partial g_k}{\partial v_i}$$

Each primitive's backward rule computes the local partial derivatives $\frac{\partial g_k}{\partial v_i}$ and multiplies them with the incoming gradient $\bar{v}_k$. This is the standard vector-Jacobian product (VJP) formulation of reverse-mode AD, which avoids materializing the full Jacobian matrices of intermediate operations. Instead, each backward rule directly computes the product of the incoming gradient vector with the local Jacobian, which is typically much cheaper and can often be expressed in closed form.

In the implementation, the transformation walks the graph in reverse order. For each `PrimitiveCall`, it looks up the corresponding backward rule stored in the primitive's `@rules["backward"]` field. This backward rule receives the original inputs and outputs of the primitive call (from the forward pass) as well as the incoming gradient $\bar{v}_k$, and returns the gradient contributions for each input. These contributions are accumulated across all uses of a value to obtain the total gradient $\bar{v}_i$.

Because the backward rules are themselves expressed in terms of primitive operations (i.e., they call `nv1_*` functions), the gradient graph is built using the same tracing mechanism as the forward pass. This means that the gradient computation is itself a Graph that can be further transformed or compiled — for example, one could in principle compute higher-order derivatives by applying `transform_gradient()` again to the gradient graph.

An important design choice is that the backward rules are stored per-primitive in the `@rules` field, rather than being hard-coded in the transformation. This makes it straightforward to add differentiation support for new primitives (see Section 2.0.5) and keeps the transformation logic generic.

Compiling a Graph Whenever we are done transforming our Graph and are ready to execute it, we need to lower it. We can convert a Graph into a `stablehlo` program via the `stablehlo()` function. The output of this is a list containing:

1. The `StableHLO` program.
2. The constant buffers in the Graph.

The output function is a regular R function, which contains the “recipe” for transforming the R function into a Graph, then lowering into a StableHLO program and finally executing it.

```
multiply_jit(x, y)

#> [1] "Hello user!"

#> AnvilTensor
#> 1
#> [ CPUf32{} ]
```

Of course, compiling the function every time the function is called would be very inefficient. Therefore, a cache is maintained that stores the executable (and constant values) for each unique combination of input types.

If we execute it again, we get the same output, but the print statement is not shown.

```
multiply_jit(x, y)

#> AnvilTensor
#> 1
#> [ CPUf32{} ]
```

If we apply it again to different input values (of the same types), we get a cache hit and directly execute the cached executable.

```
multiply_jit(nv_scalar(2, "f32"), nv_scalar(3, "f32"))

#> [1] "Hello user!"

#> AnvilTensor
#> 4
#> [ CPUf32{} ]
```

Because the compiled executable contains only the tensor operation, the print statement is not shown. If we apply the function to values of different types, we get a cache miss and the function is recompiled. This will show the print message again.

```
multiply_jit(nv_scalar(1, "i32"), nv_scalar(5, "i32"))

#> [1] "Hello user!"

#> AnvilTensor
#> 1
#> [ CPUi32{} ]
```

We can also JIT-compile functions that take in lists of tensors and output lists of tensors.

```
multiply_jit_nested <- jit(function(lst) {
  list(nv_mul(lst[[1]], lst[[2]]))
})
multiply_jit_nested(list(x, y))

#> [[1]]
#> AnvilTensor
#> 1
#> 2
#> 3
#> 4
#> [ CPUf32{4} ]
```

This will use the `in_tree` and `out_tree` fields of the Graph to flatten and unflatten the list of tensors.

The functions we JIT-compile can also take in so called “static” arguments, that can be standard R values. Below, we define a function that either multiplies or adds two values, depending on the value of the `op` argument. We need to mark the `op` argument as static.

```
add_or_multiply <- jit(function(x, y, op) {
  if (op == "add") {
    nv_add(x, y)
  } else {
    nv_mul(x, y)
  }
}, static = "op")
```

The difference between static and non-static arguments is that for the former the function is recompiled for every unique input type combination, while for the latter it needs to be recompiled for every unique value of the static argument.

```
add_or_multiply(x, y, "add")
```

```
#> AnvilTensor
#> 2
#> 3
#> 4
#> 5
#> [ CPUf32{4} ]
```

```
add_or_multiply(x, y, "multiply")
```

```
#> AnvilTensor
#> 1
#> 2
#> 3
#> 4
#> [ CPUf32{4} ]
```

Transformations like `gradient()` Just like `jit()`, the `gradient()` function takes in a function and outputs a function that contains the recipe for transforming the input function into a Graph representing the gradient of the input function.

```
add_fn <- function(x, y) {
  nv_add(x, y)
}
grad <- gradient(add_fn, wrt = c("x", "y"))
```

In order to be able to execute the gradient function, we need to wrap it in a `jit()` call. We can also jit a function that contains “standard” anvil operations and calls into the gradient function.

```
x <- nv_scalar(1, "f32")
y <- nv_scalar(5, "f32")
f <- function(x, y) {
  z <- nv_add(x, y)
  grads <- grad(x, z)
  grads
}
```

The way this works is the following: When tracing the `val_and_grad` function, a `GraphDescriptor` is first initialized. When the `add_or_multiply` function is called, it populates the `GraphDescriptor` with the inputs `x` and `y` and adds the `PrimitiveCall` to the body representing the multiplication of `x` and `y`. When we then call into the `grad` function, we create another `GraphDescriptor`, because we need a graph-representation of the of the `add_or_multiply` function so we can transform it into its gradient. After this is done, the resulting gradient graph is inlined into the parent `GraphDescriptor`.

We can see the output of this below:

```
graph <- trace_fn(f, list(x = x, y = y))
graph

#> <AnvilGraph>
#>   Inputs:
#>     %x1: f32[]
#>     %x2: f32[]
#>   Constants:
#>     %c1: f32[]
#>   Body:
#>     %1: f32[] = add(%x1, %x2)
#>     %2: f32[] = add(%x1, %1)
#>   Outputs:
#>     %c1: f32[]
#>     %c1: f32[]
```

Because this makes `f` just another graph-building function, we can wrap it in a `jit()` call and execute it.

```
f_jit <- jit(f)
f_jit(x, y)

#> $x
#> AnvilTensor
#> 1
#> [ CPUf32{} ]
#>
#> $y
#> AnvilTensor
#> 1
#> [ CPUf32{} ]
```

Debugging One of the main disadvantages of the JIT-compilation approach is that it makes the program harder to debug, because the function cannot simply be executed line-by-line. For this reason, we offer a debugging mode that allows to execute a function without wrapping it in a `jit()` call. There, the function will be executed line-by-line. However, this will mean that we don't execute the real function, but only perform the abstract evaluation, meaning you can also see the types of the intermediate values.

```
multiply_print_r(x, y)

#> [1] "Hello useR!"

#> f32{}
```

It is also possible to print intermediate tensors during program execution via `nv_print()`.

```
multiply_print_jit <- jit(function(x, y) {  
  out <- nv_mul(x, y)  
  nv_print(out)  
  return(out)  
})  
multiply_print_jit(x, y)  
  
#> AnvilTensor  
#> 5  
#> [ f32{} ]  
  
#> AnvilTensor  
#> 5  
#> [ CPUf32{} ]
```

The difference between `nv_print` and a standard `print()` call is that the former will be embedded in the StableHLO program and executed by the backend, while the latter will be executed during the tracing. The `nv_print()` function is implemented as a StableHLO custom call, c.f. the extending section.

Finally, to aid debugging, we try to provide good error messages when applying graph-building functions to the wrong types of inputs.

Static Shapes

While the JIT-compilation approach is very powerful, it has some limitations. One major restriction is that the shapes in the StableHLO programs are static and need to be known at compile time. This means that the approach does not do well when the input shapes are varying strongly. In principle, the StableHLO language does support dynamic shapes. However, the XLA compiler which we are using requires all shapes to be known at compile time.

In order to lift this restriction, it would be necessary to implement a different backend. One possible approach is to use the IREE compiler ([cite](#), [link](#)), which also consumes StableHLO programs and does support dynamic shapes. Another approach might be to lower the Graph into a different language, e.g. using the [quicksr](#) or by transpiling to Mojo. An advantage of the static shapes is that it allows for more aggressive optimizations by the compiler.

The

0.4 Examples

Despite [anvil](#) being still in a rather early stage and there are many features we would like to add, it can already be used to accelerate real-world problems. We demonstrate this with a simple logistic regression example that showcases `jit()`, `gradient()`, and `nv_while()` working together.

Logistic Regression via Gradient Descent

To illustrate how `jit()` and `gradient()` work together, we fit a logistic regression model via gradient descent using the Titanic survival dataset from base R. A more detailed version of this example is available as a package vignette.

We first prepare the data by expanding the contingency table into a design matrix and converting it to AnvilTensors.

```
set.seed(42)  
titanic_df <- as.data.frame(Titanic)
```



```

titanic <- titanic_df[rep(seq_len(nrow(titanic_df)), titanic_df$Freq), 1:4]
X <- scale(model.matrix(~ Class + Sex + Age, data = titanic)[, -1])
y <- as.integer(titanic$Survived == "Yes")
n <- nrow(X); p <- ncol(X)

X_tensor <- nv_tensor(X, dtype = "f32")
y_tensor <- nv_tensor(y, dtype = "f32", shape = c(n, 1L))

```

The model predicts survival probabilities and evaluates them with binary cross-entropy loss:

```

model_loss <- function(X, y, beta, alpha) {
  probs <- nv_logistic(X %*% beta + alpha)
  eps <- 1e-7
  probs <- nv_clamp(eps, probs, 1 - eps)
  loss <- -(y * log(probs) + (1 - y) * log(1 - probs))
  mean(loss)
}

```

We obtain the gradient of the loss with respect to the parameters using `gradient()`:

```
model_loss_grad <- gradient(model_loss, wrt = c("beta", "alpha"))
```

The training loop is implemented using `nv_while()`, which keeps the entire loop within a single compiled function. This avoids the overhead of repeatedly calling into the compiled function from R.

```

fit_logreg <- jit(function(X, y, beta, alpha, n_epochs, lr) {
  output <- nv_while(
    list(beta = beta, alpha = alpha, epoch = nv_scalar(0L)),
    \(beta, alpha, epoch) epoch < n_epochs,
    \(beta, alpha, epoch) {
      grads <- model_loss_grad(X, y, beta, alpha)
      list(
        beta = beta - lr * grads$beta,
        alpha = alpha - lr * grads$alpha,
        epoch = epoch + 1L
      )
    }
  )
  list(beta = output$beta, alpha = output$alpha)
})

```

We initialize the parameters and train the model:

```

beta_init <- nv_tensor(rnorm(p), dtype = "f32", shape = c(p, 1L))
alpha_init <- nv_scalar(0, dtype = "f32")

result <- fit_logreg(
  X_tensor, y_tensor, beta_init, alpha_init,
  nv_scalar(50000L), nv_scalar(0.1)
)

```

To verify correctness, we compare the estimated coefficients with R's built-in `glm()`:

```
#>      Parameter      anvil      glm
```

```
#> 1 (Intercept) -0.8538058 -0.8538077
#> 2   Class2nd -0.3418888 -0.3418906
#> 3   Class3rd -0.8299900 -0.8299946
#> 4   ClassCrew -0.4206345 -0.4206313
#> 5   SexFemale  0.9919736  0.9919786
#> 6   AgeAdult -0.2303528 -0.2303616
```

The estimates closely match, confirming that the gradient descent implementation converges to the same solution as `glm()`.

Further examples, including TODO, are available as package vignettes (TODO).

0.5 Extending

One of the key design goals of **anvil** is to be extendable. It is possible to add new primitive operations, transformations, and even backends.

New Primitive Operations

The simplest way to extend **anvil** is to add new primitive operations. This consists of defining a `Primitive` object, implementing the transformation rules, and creating a graph-builder function for the primitive. Below, we show how to create the (already existing) addition primitive from scratch. We start by creating the `Primitive` object that holds the transformation rules.

```
p_mul_custom <- AnvilPrimitive("mul_custom")
```

Next, we create the graph-builder function for the primitive. It needs to define the abstract evaluation rule (the `infer_fn` argument) that describes how to convert the abstract inputs into abstract outputs. Then, it appends a `PrimitiveCall` object to the `GraphDescriptor` that represents the primitive call via `graph_desc_add()`. because `graph_desc_add` returns a list of values, we unbox it and return the first element.

```
nvl_mul_custom <- function(x, y) {
  graph_desc_add(p_mul_custom, list(x, y), infer_fn = infer_binary)[[1L]]
}
```

With this in place, we can already create `Graph` objects that use the new primitive.

However, we also want to be able to translate this to `StableHLO` and compute gradients.

We can define these rules by setting the `@rules` field of the `Primitive` object. We start with the `stablehlo` rule. It takes as inputs `FuncValue` objects and returns a list of `FuncValue` object. This is exactly what the inference functions from the **stablehlo** do, so we can just use it:

```
p_mul_custom[["stablehlo"]] <- function(lhs, rhs) {
  list(stablehlo::hlo_multiply(lhs, rhs))
}
```

Next, we define the backward rule. It is expected to be a function with the following arguments:

- `inputs`: The inputs to the primitive function during the forward pass.
- `outputs`: The outputs of the primitive function during the forward pass.
- `grads`: The gradients of the outputs of the primitive function during the backward pass.
- `required`: A logical vector indicating which inputs require a gradient.

The inputs, outputs and grads are list of `GraphBox` objects and the function should return a list of the same length as the inputs. The values of this list must either be a `GraphBox` containing the gradient of the input with respect to the output (respecting the chain rule), or `NULL` if the input does not require a gradient. The required argument is a logical vector indicating which inputs require a gradient.

```
p_mul_custom[["backward"]] <- function(inputs, outputs, grads, required) {
  grad <- grads[[1L]]
  list(
    if (required[[1L]]) nvl_mul(grad, rhs),
    if (required[[2L]]) nvl_mul(grad, lhs)
  )
}
```

From this implementation we can see that any primitive operations that can be expressed in terms of `StableHLO` operations can easily be added. While this is already quite powerful, there are cases where this is not enough. For this reason, it is in principle possible to implement custom calls that perform arbitrary operations defined in C++, possibly using CUDA. This is not super ergonomic yet, but this could be improved if it shows to be necessary.

New Transformations

Because the `Graph` representation is a simple data structure consisting of inputs, outputs, and a sequence of `PrimitiveCall` objects, it is straightforward to implement new graph-to-graph transformations. The `transform_gradient()` function, which implements backward-mode automatic differentiation, is the primary example of such a transformation. It walks the graph in reverse order and applies the backward rule stored in each primitive's `@rules[["backward"]]` field to accumulate gradients.

Implementing a new transformation follows the same pattern: iterate over the `PrimitiveCall` objects in the graph, look up a corresponding rule for each primitive, and use these rules to construct a new graph. For example, one could implement forward-mode automatic differentiation by walking the graph *forward* and propagating Jacobian-vector products, or implement a graph optimization pass that fuses or simplifies sequences of primitive calls. The key design decision is that the `anvil` package does not impose any constraints on how a transformation is implemented — it only provides the `Primitive` rule mechanism as a convenient way to store per-operation information.

Custom Backends

Adding a new backend amounts to implementing a new graph-to-executable transformation, analogous to how the XLA backend translates a `Graph` into a `StableHLO` program via the `stablehlo()` function and then compiles it via PJRT. For each primitive, a lowering rule is stored in the `@rules` field (e.g., under `"stablehlo"` for the XLA backend). A new backend would define its own set of rules under a different name and provide a corresponding lowering function that walks the graph and applies these rules to produce the target representation.

As a concrete example, there is an experimental backend that compiles `Graph` objects to Fortran code via the `quicksr` package, targeting CPU execution without requiring the XLA toolchain. This demonstrates that the same graph can be lowered to fundamentally different targets. However, the process of adding a new backend currently requires some familiarity with the package internals. Providing a more streamlined and well-documented interface for registering custom backends is an area we plan to improve.

0.6 Testing

Ensuring correctness of both the forward evaluation and the gradient computation is critical for a code transformation framework. In **anvil**, we bootstrap correctness by comparing against the **torch** package wherever possible.

For forward-pass testing, each primitive's JIT-compiled output is compared against the corresponding **torch** function on randomly generated inputs. For example, to test `nv1_add`, we generate random tensors, compute the result using both `jit(nv1_add)(x, y)` and `torch::torch_add(x, y)`, and assert that the outputs match up to numerical tolerance. This is done using helper functions like `expect_jit_torch_binary()` and `expect_jit_torch_unary()`, which handle the data generation and comparison.

For backward-pass testing, the same strategy is applied to gradient computation. The **anvil** gradient (obtained via `jit(gradient(f))(x)`) is compared against the gradient computed by **torch**'s `autograd` engine. For non-scalar functions, both sides are wrapped with a summation to produce a scalar loss before differentiating, ensuring a consistent comparison.

Because **torch** is not a hard dependency of the package, the **torch**-based tests are stored in `inst/extra-tests/` and are conditionally sourced only when **torch** is installed. This keeps the test suite lightweight for users who do not have **torch**, while still running the full comparison suite in CI. For primitives that have no direct **torch** equivalent (e.g., random number generation or bitwise operations), manual tests with hardcoded expected values are used instead.

0.7 Benchmarks

Runtime benchmarks comparing **anvil** on CPU and GPU are available at <https://r-xla.github.io/benchmarks/>.

0.8 Discussion

In this paper, we introduced the **anvil** package, which is a code transformation framework for the R language. Execution is handled via JIT compilation through either the OpenXLA compiler framework or by transpiling to Fortran via **quickr**. When using the OpenXLA backend, it is possible to make use of accelerators such as GPUs. Furthermore, the package supports first-order automatic differentiation.

The biggest limitations of the current approach are twofold:

- a) Programs need to have static shapes, which means that operations such as `unique()` are currently not supported.
- b) For the OpenXLA backend, programs need to be re-compiled for each new unique input type combination.

Future work will aim at lifting the static shape and re-compilation restrictions. One solution will be to improve the support for the Fortran backend, which – as opposed to XLA – does support shape dynamism. To do this properly, the intermediate `AnvilGraph` needs to be extended to support shape dynamism, which it currently does not. Extending the Fortran backend to cover more primitives also means that the second major restriction can be lifted, at least when executing on CPUs.

Apart from these steps, there also many other next steps that would improve usability and make it more generally useful. Another important improvement is to decouple the intermediate representation (`AnvilGraph`) from `StableHLO` more thoroughly. Currently, the IR is tied too closely to `StableHLO`, which limits the ability to target alternative backends and to represent higher-level optimizations that do not map directly to `StableHLO` concepts. The simplest one is to extend the set of primitive operations supported in both **anvil** and

stablehlo. Then, more useful API functions can be added, e.g. focusing on linear algebra operations. Also, other code transformations such as higher-order derivatives, forward-mode automatic differentiation or improved vectorization transformations could be added.

Finally, one package that might be useful is to build a small modeling library on top of **anvil**, which is aimed at optimizing models via gradient-based optimization methods, e.g. for training deep learning models.

0.9 Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 460135501 (NFDI project MaRDI).

References

F. Morandat, B. Hill, L. Osvald, and J. Vitek. Evaluating the design of the r language: Objects and functions for data analysis. In *European conference on object-oriented programming*, pages 104–131. Springer, 2012. [p2]

Sebastian Fischer
LMU MunichMunich Center for Machine Learning
Department of Statistics
Ludwig-Maximilians-Universität Munich
Ludwigstr. 33, 80539 Munich, Germany
ORCID: [0000-0002-9609-3197](https://orcid.org/0000-0002-9609-3197)
sebastian.fischer@stat.uni-muenchen.de

Daniel Falbel
Posit Software, PBC

ORCID: [0000-0002-0912-0225](https://orcid.org/0000-0002-0912-0225)

Thomas Kalinowski
Posit Software, PBC

Niko German
LMU Munich

ORCID: [0009-0001-7394-8367](https://orcid.org/0009-0001-7394-8367)

Martin Binder
LMU MunichMunich Center for Machine Learning

ORCID: [0009-0008-2578-2869](https://orcid.org/0009-0008-2578-2869)

Bernd Bischl
LMU MunichMunich Center for Machine Learning

ORCID: 0000-0001-6002-6980